

PGS FIELD MANUAL v2

Protocol-Governed Systems — Cognitive Restoration Manual

Architecture · Governance · Compiler · Execution · Runtime Doctrine

Author: Bhash Ganti (aka Bachi)

© 2026 Bhash Ganti. All rights reserved. Released under the Apache-2.0 License.

Status: Public Reference Artifact — v2 · Baseline: PGS v0.8.0 · revised for the full change-management **lifecycle** (Authoring → Construction → Admission → Execution Validation → Promotion), the unified pgs_change CLI, and the Governance-Impact / Promotion→Adoption result (§4). The v0 edition is the published DOI record.

Canonical Repository: [bachipeachy/pgs_workspace](https://github.com/bachipeachy/pgs_workspace)

Audience: Architects · Compiler Engineers · Runtime Engineers · Governance Engineers · AI Coding Agents

What This Manual Is

This is a high-density architectural restoration artifact. Its purpose is to restore the correct architectural mental model of Protocol-Governed Systems in under 30 minutes — not to teach, not to document implementation, not to walk code.

Intended for: system architects, compiler engineers, runtime engineers, governance engineers, AI coding agents operating under human supervision, security reviewers, technical maintainers.

Not a tutorial. Assumes PGS familiarity. Source code, constitutions, invariants, and protocol artifacts remain authoritative. This manual restores the mental model needed to read them correctly.

AI coding agent operating constraint. PGS contains constraints that general-purpose coding agents violate unintentionally. Without architectural restoration, agents tend to introduce convenience abstractions, collapse governance boundaries, embed domain logic into runtime layers, add fallback behavior, and weaken determinism. The nine constraints below are non-negotiable before any substantial modification:

no fallback logic	no runtime discovery	no dynamic imports
no heuristic resolution	no ambient authority	no runtime topology synthesis
no short-name artifacts	compile-time governance	strict layer separation

Architectural reading rule. When ambiguity exists — governance overrides implementation convenience. This single rule resolves most architectural decisions correctly.

PGS is not: workflow orchestration with governance wrappers · a policy engine · a runtime authorization framework · a service mesh · a BPM engine.

PGS is: compile-time admissible execution construction.

Situation	Recommended Sections
New architectural enhancement	Executive Doctrine · Core Doctrine · Architectural Invariants
Compiler work	§5 Compiler Model · §3 Governance Model
Runtime modification	§6 Runtime Model · §7 Execution Model · §11 Anti-Patterns
Transport changes	§9 Transport Governance
AI agent onboarding	This section · Executive Doctrine · §4 Authoring Model · §11 Anti-Patterns
Governance extension	§3 Governance Model · §10 Federated Governance
Protocol change (new CR / governed evolution)	§4 Authoring Model

Situation	Recommended Sections
Debugging	Appendix D · §6 Runtime Model
New domain integration	Appendix E · §3 Governance Model

PGS In One Page

Protocol
↓
Compiler
↓
Snapshot
↓
Runtime
↓
Trace

Step	Statement
Protocol declares.	What may exist and how it may execute — workflow topologies, capability contracts, governance rules.
Compiler validates.	What is admissible enters the snapshot. What is not admissible is rejected at build time — it cannot be expressed, so it cannot execute.
Snapshot seals.	Compiled, attested, immutable. The runtime's only input. Never modified by hand. Never modified at runtime.
Runtime executes.	Traverses the snapshot faithfully. Contains no domain knowledge. The same engine runs blockchain, AI governance, and any future domain identically.
Trace proves.	Append-only execution evidence. Deterministic. Substrate-neutral. If an action does not appear in the trace, it is architecturally prohibited.

Three invariants that follow directly:

The runtime does not decide what may exist. The compiler already decided. The runtime does not interpret behavior. The protocol already declared it. The compiler does not execute. The runtime executes only what the compiler sealed.

The mantra:

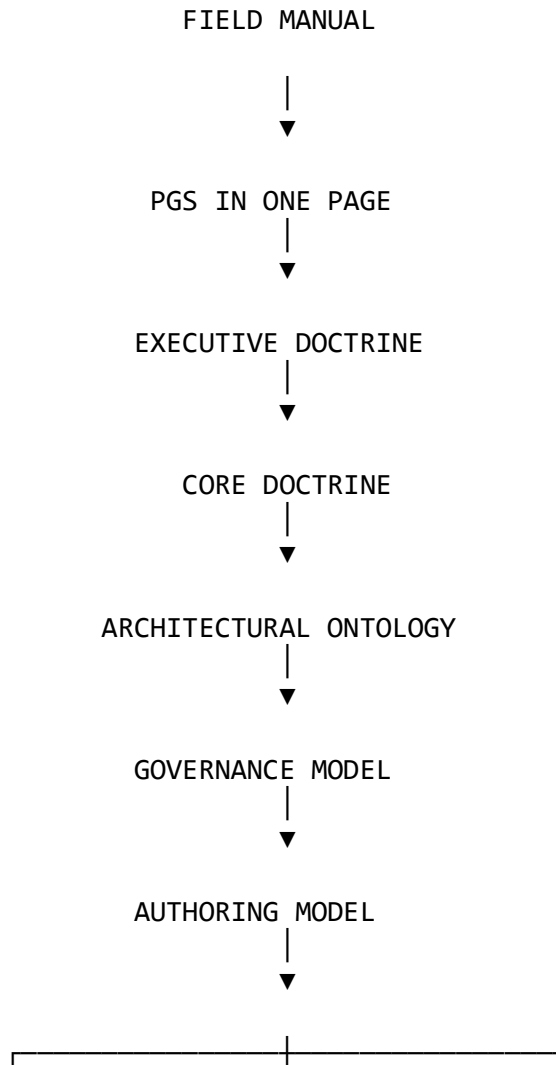
Governance defines. Compiler constructs. Snapshot seals. Runtime executes. Trace proves.

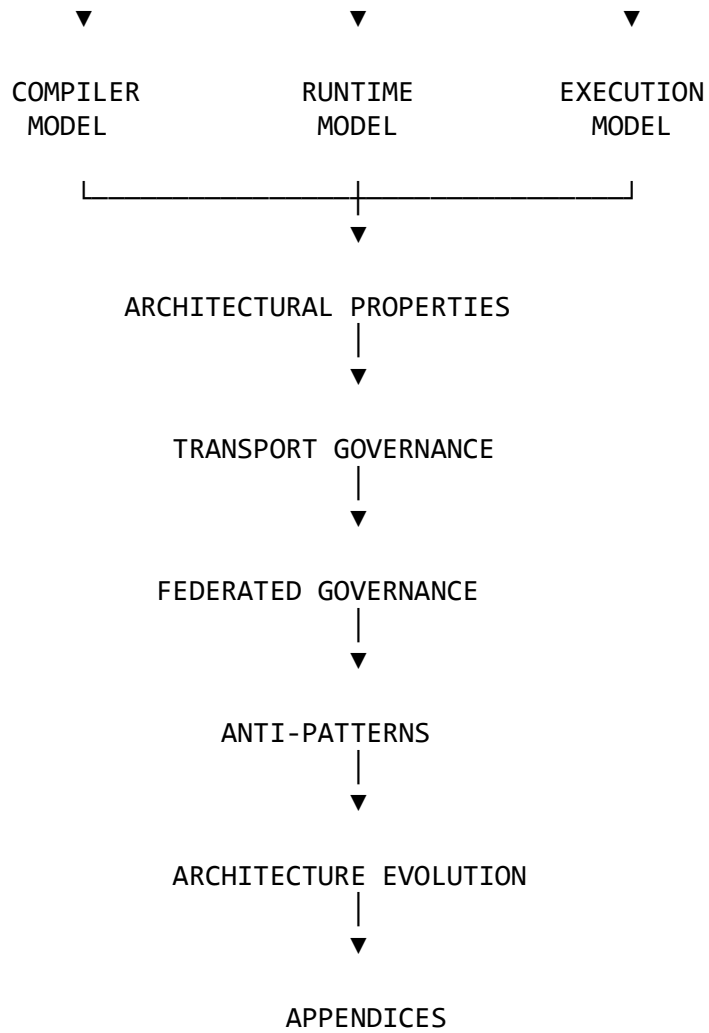
The nine execution concerns:

Prefix	Name	One-Line Role
TI_	Transport Ingress	Normalizes external input to canonical internal form
AC_	Actor Context	Binds execution authority context
IN_	Intent	Admission gate — ACK or NACK before any traversal begins

Prefix	Name	One-Line Role
WF_	Workflow	The topology — which CCs execute in what declared sequence
CC_	Capability Contract	Named DAG node — drives CT/CS pipeline, declares outcome
CT_	Capability Transform	Pure computation — zero side effects
CS_	Capability Side Effect	Controlled external state interaction — enumerated, bounded
EV_	Event	Governance signaling — emitted as declared values
TE_	Transport Egress	Projects internal result to external form
RB_	Runtime Binding	Maps declarations to implementations (not authority)

Manual reading map:





0. Executive Doctrine

One-line summary: Behavior is a compiled artifact of protocol declarations — not an emergent property of implementation code.

Principle	Statement
Protocol Sovereignty	Protocol is the sole source of behavioral truth
Runtime Dumbness	The runtime has zero domain or business logic knowledge
Compile-Time Resolution	All behavior determined and validated before execution
Zero Inference	Nothing is assumed, guessed, or defaulted
Fail Hard	No fallback, no recovery, no silent failure
Declaration Over Refactor	Change behavior by changing declarations, not code
Governance Before Execution	Protocol is law; execution is enforcement; trace is proof
Determinism	Identical inputs → identical traces, always
Security by Construction	Unauthorized behavior is never constructed, not merely blocked
Orthogonality	Governance surfaces evolve independently; coupling between surfaces is a violation
Snapshot Sovereignty	Runtime executes compiled state exclusively; no live modification

1. Core Doctrine

1.1 Protocol is the Source of Truth

- Behavior identity is carried by protocol artifacts, not code
- Business logic is encoded in protocol artifacts, not in implementation code
- Code may be regenerated, replaced, or machine-authored; governance remains stable
- Behavioral authority persists independently of implementation lifecycle

1.2 Runtime Dumbness (Semantic-Agnostic Execution)

- The runtime enforces graph structure — it does not interpret domain meaning
- Same execution engine runs blockchain, AI governance, and collatz conjecture identically
- No domain logic is permitted in the runtime layer

1.3 Compile-Time Resolution

- Execution graphs are constructed before any runtime begins
- Behavioral admissibility is determined at compile time, not runtime
- If the compiler does not construct the path, the implementation cannot traverse it

1.4 Zero Inference

- No implicit defaults, no heuristics, no filesystem scanning
- All artifact locations must be declared explicitly
- No `../` traversal, no `cwd()`, no environment sniffing

1.5 Fail Hard

- Missing artifact → hard failure, not silent skip
- Constraint violation → reject, not log-and-continue
- No graceful degradation that masks architectural violations

1.6 Deterministic Execution

- Execution is a pure function: $\Phi(G, \text{input}, \text{actor_context}) \rightarrow (\text{result}, \text{trace})$
- Replay is structural, not reconstructed

1.7 Governance Before Execution

- Constitutions define what is admissible
- Compiler validates — only conformant behavior enters the snapshot
- Runtime enforces — only snapshot behavior executes

1.8 No Ambient Authority

- Code has no inherent authority derived from its execution context

- All authority originates exclusively from protocol declarations: (AC, IN, WF, CC)
- An operation has exactly the authority declared in its artifacts — no more
- Confused deputy attacks are structurally eliminated, not defended against

1.9 Snapshot Sovereignty

- Runtime executes compiled snapshot state exclusively
- Snapshot is immutable during execution — no live modification
- Runtime behavior cannot diverge from compiled admissibility state
- To change behavior: change protocol source → recompile → rebuild snapshot

1.10 Architectural Compression

- PGS intentionally compresses architecture into a small number of explicit executable concerns
- A smaller ontology with stronger invariants is preferred over a larger ontology with heuristic flexibility
- This reduces governance ambiguity, hidden behavioral surfaces, cognitive reconstruction cost, and AI-agent violation probability
- The closed set of nine execution concerns and five governance surfaces is a feature, not a limitation
- Expanding ontology size without governance necessity is treated as architectural debt

2. Architectural Ontology

2.1 The Nine Execution Concerns

Prefix	Name	Role	Group
TI_	Transport Ingress	Normalizes external input → canonical internal form	Boundary
AC_	Actor Context	Binds execution authority context; establishes admissible principal identity	Authority
IN_	Intent	Declarative admission gate; validates payload before graph traversal	Authority
WF_	Workflow	Declarative execution topology graph; compile-time governed traversal; determines admissible step sequences	Authority
CC_	Capability Contract	Named DAG node; declares inputs, outputs, routing outcomes	Capability
CT_	Capability Transform	Pure computation; deterministic; zero side effects	Capability
CS_	Capability Side Effect	Controlled external state interaction; enumerated, bounded	Capability
EV_	Event	Control plane + observability; governance signaling; active, not passive	Observation
TE_	Transport Egress	Boundary projection only; formats internal results for external systems	Boundary

Orthogonal Resolution:

Prefix	Name	Role
RB_	Runtime Binding	Maps CT_ and CS_ declarations to concrete implementations; provides execution mapping, not authority

2.2 Governance Constructs

Construct	Role
Constitution	Defines invariants for an artifact domain
Invariant	Structural law enforced at compile time
Assertion	Specific check instantiated from an invariant
STRUCTURE	Compiler build configuration; governs scope of compilation
FQDN	domain::ARTIFACT_CODE_Vn — canonical artifact identity; no short names
Snapshot	Sealed, read-only compiled output; the runtime's only input

Construct	Role
Federation Boundary	Declares a distinct semantic governance authority in <code>pgs_governance/registry/FB_*/</code> ; one sovereign boundary (<code>FB_CONSTITUTION</code>), all others delegated

Version immutability: Versions are immutable. There is no “latest.” A change to behavior requires a new version number (`_V1`, `_V2`, etc.) — the old version remains valid and unchanged in the snapshot.

2.3 Governance Surfaces

PGS has five orthogonal governance surfaces. Each governs a distinct concern. They evolve independently — adding governance sophistication to one surface does not couple to any other. New governance surfaces may be added as long as the separation-of-concerns principle is not violated by design.

Governance Surface	Governs
Identity	What entities ARE — actor declaration, FQDN, artifact identity
Authority	What entities MAY DO — admissibility, role grants, admission gates
Execution Topology	Admissible traversal — which capabilities execute in what declared sequence; immutable after compilation
Execution	Computation and side-effect semantics — CT purity, CS boundary, output contracts
Boundary	Admission and projection — TI normalization, TE rendering, transport orthogonality

Key invariant: Governance surfaces are orthogonal planes. Topology governs traversal structure only. Authority governs eligibility only. Neither governs the other. Execution is semantically blind to both.

2.4 Artifact Code Prefixes

Prefix	Meaning	Prefix	Meaning
WF_	Workflow	CC_	Capability Contract
CT_	Capability Transform	CS_	Capability Side Effect
IN_	Intent	RB_	Runtime Binding
EV_	Event	AC_	Actor Context
TI_	Transport Ingress	TE_	Transport Egress
STRUCTURE_	Build configuration		

3. Governance Model

3.1 Constitutions

- Each artifact family governed by a constitution
- Constitution declares invariants that all artifacts of that type must satisfy
- Violation → compiler rejects the artifact; it never enters the snapshot

3.2 Invariants

- Structural laws applied at compile time
- Examples: FQDN identity required, no undeclared outcomes, no missing bindings
- Invariants are not advisory — violations are hard failures

3.3 Assertions

- Specific checks instantiated from invariants
- Registered in handler registry; resolved at compile time
- Static imports only — no dynamic loading of assertion handlers

3.4 Protocol Surface Closure (Vocabulary Closure Invariant)

- Executable Behavior $\subseteq$ Expressible in V (the declared concern vocabulary)
- Behavior not expressible in V cannot pass governance, cannot enter snapshot, cannot execute
- This is a construction-time property, not a runtime guard

3.5 Governance Dividend

Named Property: As governance surface matures, cost-of-change decreases rather than increases.

- Conventional: low initial governance cost → debt accumulates → per-change cost grows unboundedly
- PGS: higher initial governance cost → debt does not accumulate → per-change cost remains stable
- Complexity: $O(N + M)$ vs conventional $O(N \times M)$ where N = capabilities, M = workflows
- Attack surface: $|CS_| + |AC_| + |RB_|$ — finite and enumerable

PGS separates implementation authorship from behavioral admissibility. Behavioral admissibility remains compiler-governed regardless of whether implementation is human-authored or machine-authored. AI may generate implementation, transforms, and workflow proposals — but AI does not define admissibility. Governance defines admissibility. Compiler constructs it. Runtime enforces it. Trace proves it.

3.6 Federation Boundaries

The governance registry is organized into **federation boundaries** — named semantic governance authorities, not filesystem folders.

Current boundary inventory:

Boundary	Level	Governance Scope
FB_CONSTITUTION	Sovereign	Root protocol semantics, governance meta-rules, artifact identity, FQDN
FB_TOPOLOGY	Delegated	Execution topology, WF/CC legality, CT/CS surface closure, routing, binding
FB_TRANSPORT	Delegated	Ingress/egress semantics, transport boundary rules, admission
FB_AUTHORITY	Delegated	Actor authority, execution admissibility, authority state
FB_VOCABULARY	Delegated	Protocol terminology, execution state vocabulary
FB_CONFORMANCE	Delegated	Test data, conformance assertion rules
FB_IDENTITY	Delegated	Actor identity semantics, identity/authority separation
FB_BLOCKCHAIN	Delegated	Blockchain domain build configuration
FB_AI_GOVERNANCE	Delegated	AI governance domain build configuration
FB_EXECUTION_SCHEDULING	Delegated	Parallelism, synchronization, deterministic joins
FB_EXECUTION_PLACEMENT	Delegated	Execution locality, substrate placement, runtime deployment admissibility
FB_SECURITY_DOMAIN	Delegated	Classification domains, compartmentalization, secure information-flow legality
FB_CRYPTOGRAPHIC_TRUST	Delegated	Snapshot signing, attestation, encryption, trust admissibility
FB_CHANGE_MGMT	Delegated	Governed evolution — the change-management pipeline, dossier lifecycle, stage purity, gates (§4)

FQDN namespace: All governance artifacts use `fb.<boundary_lower>::ARTIFACT_CODE` format.

```
fb.constitution::CONSTITUTION_FEDERATION_BOUNDARY_V0
fb.topology::CONSTITUTION_EXECUTION_TOPOLOGY_V0
fb.constitution::STRUCTURE_BUILD_PLATFORM_CONFIG_V0
```

Sovereignty model: Exactly one sovereign boundary (FB_CONSTITUTION). All others are delegated. A second sovereign boundary is a constitutional violation.

Governance locality doctrine: A governance artifact belongs in pgs_governance (central) if and only if it governs all PGS systems universally. Domain-specific governance law belongs in the domain repository.

Anti-sprawl rule: Federation boundaries must not be created to mirror organizational structure, repository topology, or implementation convenience. A boundary exists only when a distinct semantic governance authority exists.

Authoritative doctrine artifact:

fb.constitution::CONSTITUTION_FEDERATION_BOUNDARY_V0

4. Authoring Model — Governed Evolution (Change Management)

One-line summary: Evolution is itself a governed protocol concern — every protocol change travels a governed **lifecycle** from Change Request through Authoring, Construction, Admission, Execution Validation, and Promotion, leaving a complete evidence chain (the dossier). The Authoring Mandate is a milestone in that lifecycle, not its end.

Governing boundary: FB_CHANGE_MGMT (delegated, in pgs_governance/registry/).

Implementation: pgs_change_mgmt repo — stage templates in change_mgmt/templates/, dossiers in change_mgmt/dossiers/<domain>/<subdomain>/, lifecycle engine in pgs_change_mgmt/engine/ fronted by one CLI, pgs_change (\$4.11).

The closing of the loop: PGS governs construction (compiler) and execution (runtime). The change pipeline governs change itself. If the Protocol Snapshot does not change, the system is invariant by definition — the loop closes only when a new snapshot is **admitted, execution-validated, promoted into the owning source registries, and recompiled to VALID**. Change-management output is not code; it is an **admitted protocol delta** whose promotion the compiler gates.

4.1 The Pipeline (one CR = one dossier)

Stage	Artifact	Answers	Key Rule
1	change_request_<sub>_v0.md	Classification + Problem · Outcome · Known Facts · Deferrals	Business language only; baseline claims verified against snapshot, never memory
2	domain_model_<sub>_v0.md	Entities · Processes · Baseline fit · Gaps	Record what was <i>searched</i> , not only what was found
3	analysis_loop_<sub>_v0.md	Gap resolutions, iterated to Discovery Saturation	Every answer carries evidence; overturned answers are marked, never erased
4	business_model_<sub>_v0.md	Canonical consolidation (all downstream stages project from this)	Consolidation, not re-litigation

Stage	Artifact	Answers	Key Rule
4b	(Section 7 of BM)	Authoring Scope: IN this CR vs deferred	Not listed = not built
5	business_intent_<sub>_v0.md	WHAT — behavior, objects, identity, invariants, actions	Provisional capability names OK; no bindings/paths
6	governance_intent_<sub>_v0.md	WHERE — domain/subdomain, ownership, dependencies	No new artifact codes; cross-subdomain writes forbidden
6b	design_intent_<sub>_v0.md	HOW — FQDNs, topology, schemas, stores, module paths, RBs	Gate 1 — Design Approval (full dossier reviewed as a body)
7	authoring_mandate_<sub>_v0.md	IN WHAT ORDER — topologically sorted build waves	Mechanical derivation; must reconcile with 6b exactly. Gate 2 — Mandate Approval (dossier locked)
8	build_sheet_<sub>_v0.md	Construction projection — governed design assembled into a per-artifact Build Sheet Set (no new design)	Assembled not authored; governed by CONSTITUTION_CONSTRUCTION_V0
9	construction_record_<sub>_v0.md	Construction evidence: built artifacts, compiler/runtime results, deviations, discoveries	Evidence only; CR closes after the Record is complete and artifacts compile clean

Every dossier stage (1–7) is a **structured register document**: the actor emits register rows, a deterministic renderer owns the document, and a **structural oracle** validates it mechanically (well-formed FQDNs, controlled vocabularies, per-row traceability, cross-stage code reconciliation) before human review. The **design/authoring pipeline (Stages 1–7)** ends at Stage 7. Everything after Gate 2 — Stage 8 Build Sheet Set → **construct** → **admit** → **execution-validate** → **promote** → Stage 9 Construction Record — is a *distinct governance authority*, governed by `fb.change_mgmt::CONSTITUTION_CONSTRUCTION_V0`

(rationale: pgs_change_mgmt/doc/CONSTRUCTION_MODEL_V0.md). Design is open and human-gated; construction is non-authorial transcription in which the builder decides nothing. The dossier stages (1–9) are the *authored artifacts*; the **lifecycle states** below are the *governed status* a change passes through — a separate axis (§4.1a).

Discovery Saturation (Stage 3 stop condition) — all three simultaneously: no unresolved CRITICAL gaps · no open analyst questions · no dependency expansion in the last pass.

4.1a Lifecycle States — the status axis

The dossier stages record *what was authored*; the lifecycle records *how far the change has travelled toward a promoted snapshot*. A change is never “done” because a document exists — it is done when it reaches PROMOTED and the recompiled snapshot is VALID. Each transition is gated and emits evidence that lives **outside** the read-only snapshot.

DRAFT —author (S1–S7)→ (mandate locked, Gate 2)
 —construct (S8)→ CONSTRUCTION_COMPLETE design lowered to a candid
 ate protocol delta
 —admit→ ADMITTED_UNVALIDATED Compilation Unit compiles;
 impls may not yet exist
 —validate→ EXECUTION_VALIDATED CR's acceptance scenario
 runs + observes = PASS
 —promote (S9)→ PROMOTED delta copied to owning re
 gistries; compiler is the gate

State	Reached by	Gate to leave it	Evidence
DRAFT	opening a CR	Gate 1 (Design) · Gate 2 (Mandate) — human-closed	the dossier (Stages 1–7)
CONSTRUCTION_COMPLETE	construct (build)	Construction-Completeness: zero gap census	Build Sheet Set + cr_ir/
ADMITTED_UNVALIDATED	construct (admit)	Admission: the Compilation Unit (Baseline U Generated Delta U Supplementary) compiles clean	placement_manifest.json, governance_impact.json
EXECUTION_VALIDATED	validate	Execution Validation: the CR's declared acceptance_scenario executes and every observation matches	validation_report.json
PROMOTED	promote -- confirm	the compiler admits the promoted registries (rollback-all-on-red)	promotion_report.json

Admission is a first-class stage, distinct from execution. A change can be *admissible* (its protocol delta compiles) long before its CT/CS implementations exist — that snapshot is marked ADMITTED_UNVALIDATED, never VALID. Admission proves **Protocol Completeness**; Execution Validation proves the behaviour actually runs. Conflating the two is the classic “it compiles, ship it” error the lifecycle refuses to make.

4.2 Purity Ladder

Each stage admits exactly one more vocabulary class. Exception at every rung: artifacts that already exist in the baseline may be cited by exact FQDN as evidence — citing the baseline is observation, not design.

Stages 1–4	business language only (nothing new gets a code)
Stage 5	+ provisional capability names (IN/WF/CC vocabulary, unbound)
Stage 6	+ placement (WHERE) – still no new codes
Stage 6b	+ binding FQDNs, topology, schemas, stores
Stage 7	+ build order

4.3 Elicitation Contract + Stage Execution Rules

Every stage template opens with **questions for the human, each paired with declared intent** (how the answer will be used). Unanswered question = open gap, never license to assume. The human supplies governed knowledge; the agent structures, verifies, and projects it.

Every stage carries **execution rules** — accumulated failure knowledge folded back from completed CRs:

Rule	Origin
Verify every baseline claim by reading the snapshot — never from memory	Agents trust recall; recall is stale
Search the inventory before authoring any new EV_/CT_/CC_	Needed artifacts often already exist (e.g., an event reserved for exactly the CR at hand)
EV_ records facts; it never triggers execution — workflow chaining is a gateway CC	Event-subscription designs cannot compile or run
WF nodes are IN / CC / EXIT only — a WF never appears inside a WF	Sub-workflow invocation = CC_INVOKE_* on CS_WORKFLOW_GATEWAY_V0
A store is written only by its owning subdomain's CCs	Peer-store writes = dependency-gap CC owned by the peer, triggered by the CR
All codes carry _V<n>; counts reconcile across 6b/7	Unversioned codes and drifting counts are defects

4.4 Canonical Documentation Set (the change agent's oracle)

Complete, implementation-free context for any change agent — human or automated:

1. Protocol Snapshot	what the system currently does (runtime-executable)
2. PPS Snapshot	full compiled inventory (pps_snapshot/index.json) – check before authoring
3. Field Manual	this document – operational doctrine

4. Concept Papers architectural rationale (why)
+ the dossier built so far

Agent context is declared in

pgs_change_mgmt/change_mgmt/templates/0_agent_context_template_v0.md — the reading assignment loaded at CR start alongside 1_change_request_template_v0.md; extend awareness by adding rows, not code.

4.5 Governance Dividend in Practice

Each cycle's manifest carries methodology lessons forward into the stage templates. Templates are the accumulation vehicle: every future CR (and agent) inherits prior failures as enforced rules, not folklore. Seven CRs executed to date (consensus_pos · block · data_model · consensus_propose · mempool · orchestration · chain) — the conceptual model is documented in the change-management concept paper (fifth in the series).

4.6 Authoring Protocol — Machine-Block Artifacts

When the change agent (human or automated) authors a machine-block artifact (CC / WF / IN / RB / CT / CS / STRUCTURE), three rules govern *how*:

- **Structured Contract → Renderer.** The agent emits a validated **structured contract object**, never free-form machine syntax; a deterministic renderer converts that object into the artifact's machine block. Free-form machine-syntax authoring is non-conformant — the experiment showed the representation, not the model, was the ceiling. (Reference impl: `contract_renderer.py`.)
- **Generate, Never Patch (b').** The agent *generates* an artifact from the mandate; it never patches an existing one. On compiler rejection it regenerates fresh from the mandate plus the compiler diagnostics — diagnostics are context, not a diff to apply. Artifacts are disposable output; the **mandate is authoritative**. Authority stays upstream.
- **Agent Role.** The agent drafts; pi and the compiler govern; the human approves and commits. The worker is replaceable and authority never resides in it — this is what makes `COMPILER_VALIDATED_CLOSURE` (change-mgmt constitution §3) sound: artifacts are correct only when the compiler admits them, human review advisory.
- **Authority Invariance.** The pipeline is *authority-invariant with respect to the authoring actor* — the same templates, projection contracts, structural oracle, and compiler rules govern a human, a local model, or a frontier model identically (shown empirically by running one CR under Qwen 3.5 and Claude Opus 4.8 against an identical governance scaffold: drafts and convergence differed, governance did not). The actor possesses **no governing authority**: it cannot unilaterally alter governed state and cannot admit a change — only the compiler admits, only the oracle clears a stage, only a human closes a gate. *The actor proposes; governance disposes*. Authoring is interchangeable; authority is not.

4.7 Evaluation Doctrine — Judging Authored Output

How authored output (and the agent producing it) is judged is itself governed — the evaluator is not exempt from rigor:

- **Identity-Preserving Taxonomy.** Classify every artifact reference by resolving its identity against `artifact_index/index.json` before judging: **A** exact · **B** typo-alias · **C** wrong-domain · **D** proposed-new · **E** fabrication. Only **E** (no identity anywhere) is a hallucination; A–D all preserve identity. Aggregate not-found counts are inadmissible — they over-flag legitimate new-design FQDNs. (Reference impl: `grounding_audit.py`; mirrors constitution rule `IDENTITY_PRESERVING_REFERENCE_VALIDATION`.)
- **Trace Beats Aggregates.** A load-bearing claim requires end-to-end artifact-identity tracing. Aggregate counts, query tallies, and regex matches are insufficient evidence for any conclusion doctrine will rest on.
- **System Boundary.** The harness and the evaluator are part of the system under test. Distinguish **worker** vs **harness** vs **evaluator** variation before attributing a finding — e.g. ollama silently front-truncating at the ~4k default `num_ctx` is a harness fault, not a worker failure. Never encode worker-specific traits (verbosity, discovery depth, run-to-run variance) as doctrine — see *Research Classification* below.

4.8 Authoring Transports — the Trifecta

The actor-invariance of §4.6 is realized as **three interchangeable transports over one worker interface** (`execute_stage(StageInput) → StageOutput`). Only the transport between stage-prompt and worker differs; validation, handoff, gates, and figure-of-merit are identical.

Transport	Worker	How
Automated	a model (local or frontier)	drives the stage in a tool-loop, grounding via <code>pi</code>
Guided	a human — or Claude Code, a <i>grounding-capable</i> worker with <code>pi/Bash</code> in-session	export a governed Stage Package (<code>run_interactive --export</code>), paste the reply into <code>response.md</code> , <code>--import</code> ; the engine ingress-validates at the Human Mutation Boundary , then runs the identical engine
Offline replay	a recorded response	re-run deterministically

Two rules keep the transports honest: - **Worker Isolation.** A worker authors from the Stage Package + prior handoffs + `pi` + gate feedback **only** — never from platform (oracle/gate/compiler) source. Worker Mode is binary: *Package-only* (valid) vs *Used platform internals* (contaminates the interchangeability measurement). - **Worker Conformance ≠ model quality.** A worker that emits an invalid governed projection or skips

required grounding is *correctly rejected* by the compiler/oracle — a **Worker Conformance** failure, not evidence “the model is weak.” The compiler is the stable component; the worker is the variable one. *The actor proposes; governance disposes.*

Knowledge Partition (the framing that makes this sound): **PI = what is · CR = what is desired · Worker = how to transform what-is into what-is-desired.** The worker owns *disposition* (how); it never owns *evidence* (what-is, supplied by the platform) or *desire* (what-is-desired, supplied by the human through the seed).

4.9 Governed Authoring Properties

Each is a mechanically-checked property, not a review convention — the pipeline can no longer silently lose it: - **Authoring Completeness (Boundary 2 — “you cannot compute authority from meaning”)**. A human-owned, non-derivable input (e.g. a subdomain’s Purpose) must be supplied by a human *before its first use*, or the stage halts **AUTHORING_INCOMPLETE** — the pre-stage dual of ingress NACK. A seed gap *is* an authoring gap, caught at the earliest phase. - **Belief Preservation = k/N**. A worker cannot silently drop a previously-VERIFIED belief between stages: the oracle detects the omission → the worker re-queries pi and re-establishes evidence → the oracle confirms closure. Belief continuity is governed. - **Projection Fidelity**. Each stage’s authoritative JSON handoff must project *identically* to its markdown (no dropped rows, no missing required register). The gate that would have caught the original lossy baseline stands permanently. - **Design Review Contract**. Every stage certifies **Part A** (engine-certified readiness, classified unknowns) and advertises a bounded **Part B** (human-engagement — decisions pi/governance genuinely cannot answer; questions only). A human’s Part-B answer re-enters *only* through the seed (business truth), after which the pipeline replays forward; the agent never injects it mid-pipeline. - **Execution Validity (post-construction)**. A promoted change must not merely *compile* — its declared acceptance_scenario must *execute and observe as expected*. This is enforced as the EXECUTION_VALIDATED lifecycle gate (§4.11), the mechanical dual of admission: Admission proves Protocol Completeness, Execution Validation proves the behaviour runs.

4.10 The Construction Compiler (Stage 8)

Construction is a **second compiler** — deterministic, no worker, no authorship — that lowers the locked mandate into protocol artifacts. Its input is the **cr_ir/** set: the per-stage governed handoffs (cr_ir/<stage>.json), the authoritative **Construction Projection S8** consumes (the stage markdown is human-facing; the JSON is the source of truth downstream).

- **Contracts own semantics; exactly one authority.** New capabilities enter as *candidate capability contracts* in a single **Compilation Unit** beside canonical ones — the compiler cannot tell candidate from canonical and applies identical rules (the former “D4” second authority dissolves rather than relocating).

- **Pipeline:** Project → Lower (execution expansion · binding propagation · store join · type propagation) → Validate → Serialize. The constraints are the gap census; the renderer only formats.
- **Propagate, never invent (TYPED_PORT as forcing function).** A port type is either declared on a contract Interface or propagated from one; an unresolved port is reported TYPED_PORT / GAP_DOSSIER, never guessed. The fix belongs at the authoring source — regenerate the stage, never patch the artifact.
- **Persistence is a binding, not a port.** A CS store access is a Store-Join ACCESSES edge (from the capability's storage_type + STRUCTURE), never a typed dataflow port; TYPED_PORT never models storage. (Full model + rationale: pgs_change_mgmt/doc/CONSTRUCTION_MODEL_V0.md.)

The construct step ends at **admission**: the generated delta is assembled with the canonical surface into a single **Compilation Unit** — Baseline ∪ Generated Delta ∪ Supplementary Artifacts (origin-agnostic; the compiler cannot tell generated from canonical) — and handed to the compiler. If it compiles, the change reaches ADMITTED_UNVALIDATED and the ownership decision is frozen once as a placement_manifest.json beside the finale set. Admission is *compile-time* completeness; it does not require the CT/CS implementations to exist yet.

4.11 Execution Validation (the validate stage)

Admission proves a delta *compiles*; Execution Validation proves it *runs as the CR intended*. It is **generic and CR-transparent**: the CR owns the *expectation*, the engine owns the *mechanism*.

- **The CR declares an acceptance_scenario** in its dossier (a business objective + its executable realization). The validation engine has **zero domain knowledge** — it never guesses a workflow, payload, or success criterion.
- **Mechanism:** build a fresh candidate snapshot → execute the declared scenario's workflow steps → observe → compare against declared expectations → emit validation_report.json. An observation may only *resolve an address and compare a value* over the **protocol surface only** — \$.steps.<id>.surface.<path> (a field of a step's result surface) or \$.steps.<id>.store.<store>.<path> (state of a declared store after a step; .count on a list). No trace parsing, no internals.
- **Verdict gates the next stage:** EXECUTION_VALIDATED on PASS, EXECUTION_INVALID otherwise. Promotion refuses without PASS evidence. The scenario itself is admissibility-checked (VALIDATION_SCENARIO_SCHEMA) before it runs — a malformed expectation is rejected, not silently passed.

4.12 Promotion, and Governance Impact (the promote stage — Promotion ≠ Adoption)

Promotion (Stage 9) is the only step that writes governed source. It is deliberate and hard to reverse, so it is gated three ways and is **all-or-nothing**:

- **Validation gate.** Refuses unless the change is `EXECUTION_VALIDATED`.
- **Governance-Impact gate.** During construction the CR *discovers* the exact surface additions it needs and records them descriptively in `governance_impact.json` — **discovery, not authority**. At promotion, if the canonical surface does not yet admit every required addition, promotion is **BLOCKED** with the missing FQDNs named. A CR *discovers the governance changes it needs; it never performs them*. The governance authority approves them separately by adding them to the canonical surface. **Promotion never writes governance.**
- **Compiler gate.** With `--confirm`, the finale artifact set is copied into each **owning source registry** (routed by the Placement Manifest — a cross-repo delta lands in each owning repo; **never** the read-only `protocol_snapshot/`), then the compiler recompiles. Red $\Rightarrow$ rollback-all, registries unchanged. Green $\Rightarrow$ `promotion_report.json` (the S9 closure certificate, symmetric with the validation report).

Promotion $\neq$ Adoption. Promotion makes a delta part of the governed source. *Adoption* — a platform deciding to evolve onto a promoted capability — is a separate, upstream authority (`platform_delta_approved.json`) that governance approves and that sits ahead of promotion. The CR surfaces impact; governance disposes; promotion only gates. This keeps a single rule intact end-to-end: **the actor proposes, governance disposes.**

4.13 The `pgs_change` CLI — one executable, the lifecycle as verbs

The whole lifecycle is fronted by a single executable whose four verbs *are* the state transitions of §4.1a. The CLI contains no authoring, construction, validation, or promotion logic — only dispatch — which is exactly what lets the mechanics stay governed and interchangeable. Compiler mechanics (`build` / `admit` / `persist`) are internal to construct, not user verbs.

Verb	Transition	What it does
<code>pgs_change author [--guided] --worker ... [--stage N]</code>	DRAFT $\rightarrow$ mandate (S1–S7)	drives the dossier stages over the trifecta transports (§4.8) $\rightarrow$ <code>cr_ir/</code>
<code>pgs_change construct --projection ... --domain ... --subdomain ... [--persist DIR]</code>	$\rightarrow$ ADMITTED_UNVALIDATED	lowers <code>cr_ir/</code> to a delta (build), then forms the Compilation Unit and asks the compiler (admit)
<code>pgs_change validate --dossier <domain>/<sub></code>	$\rightarrow$ EXECUTION_VALIDATED	runs the CR's acceptance_scenario against a fresh candidate snapshot
<code>pgs_change promote --dossier ... [--from DIR] [--registry-root DIR] [--confirm]</code>	$\rightarrow$ PROMOTED (S9)	validation + governance-impact gated; copies the finale

Verb	Transition	What it does
		set into owning registries; compiler is the gate

PGS_WORKSPACE (absolute) is required — the lifecycle never guesses it from cwd. Without --confirm, promote is a dry-run that reports readiness and stops short of writing any registry.

5. Compiler Model

5.1 Pipeline Stages (in order)

Stage	Name	What Happens
S1	EXTRACT	Artifact files located via STRUCTURE_ build config; deserialized into typed PIR structures; no filesystem scanning
S2	CANONICALIZE	Artifacts normalized; edge typing resolved; canonical representation established
S3	SEMANTIC_ADDRESSING	Semantic addresses allocated for all graph entities; address space closed
S4	GOVERN	Invariants evaluated via registered assertion handlers; topology closure, step identity, canonical surface, authority/transport orthogonality, and graph completeness all checked
S5	CONSTRUCT	Execution topology constructed; CT/CS IR built; CC projections assembled; bindings resolved; topology sealed
S6	PROJECT	Visualization projections generated (graph JSON, PNG rendering)
S7	MATERIALIZE	Compiled artifacts written to snapshot directories; evidence graph written to evidence_snapshot/; trace events flushed
S8	VERIFY	All materialized artifacts verified on disk; hash integrity checked; evidence graph integrity validated
S9	ATTEST	Cryptographic attestation computed and written for each structure and the full snapshot

5.2 Compiler Constraints (Non-Negotiable)

- No execution during compilation — compiler never runs CT/CS implementations
- Static imports only — no dynamic module loading
- Deterministic output — same source → same snapshot, always
- No cross-artifact inference — each artifact is validated against its own schema + invariants
- Handler registry must be fully populated before compile starts
- Compiler validates and rejects — it never repairs malformed protocol state

5.3 Two Phase Types

Phase Type	Name	Semantics
Phase Type A	Per-Structure Local	DISCOVER → MATERIALIZE within a single structure; single-structure semantic closure
Phase Type B	Cross-Structure Aggregation	Consumes declared output surfaces from multiple structures; produces federated governance products

Phase Type B is governed by an aggregation STRUCTURE_ artifact with an aggregation_type field. The CLI routes on this field to the appropriate aggregation runner.

Phase Type B invariants — non-negotiable: - Deterministic, declarative, bounded, structure-declared - No implicit dependency ordering between structures - All contributing source dirs declared explicitly in artifact_source_dirs - Must run after all contributing Phase Type A builds complete

Current Phase Type B families:

Aggregate	aggregation_type	Status
Vocabulary	VOCABULARY	Implemented
Transport	TRANSPORT	Planned
Authority	AUTHORITY	Planned
Conformance	CONFORMANCE	Future

5.4 Compiler Evidence Graph

The compiler emits evidence_graph.json per structure during S7. This is the compiler's governed observability artifact — a semantic causality graph over all S1–S7 trace events, analogous in role to the execution trace for the runtime.

Field	Content
events	All compiler trace events (S1–S7), typed by operation, tagged by family
edges	CAUSALITY edges + STAGE_SEQUENCE edges
families	Events bucketed by semantic concern (DISCOVERY, TOPOLOGY, ADDRESSING, GOVERNANCE, CONSTRUCTION, PROJECTION, MATERIALIZATION)
evidence_graph_hash	SHA-256 over core content; verified by S8
structure_id, compiler_version	Envelope metadata — excluded from hash

Formal guarantees:

Guarantee	Statement
Causality definition	Two inferred patterns: (1) S1 discovery_complete → all subsequent node_created events; (2) S5 all ct_ir_built + cs_ir_built + cc_projection_built → construction_complete
Stage ordering	Exactly 7 STAGE_SEQUENCE edges (S1→S2→S3→S4→S5→S6→S7); deterministic given same source
Edge kind exhaustiveness	Exactly two edge kinds: CAUSALITY and STAGE_SEQUENCE. No others are permitted. Consumers may assert this.
Graph determinism	Same source + same compiler version → bitwise-identical events, edges, families content
Projection completeness	All events from all stages S1–S7 present; no stage selectively omitted
Hash contract	SHA-256 over {event_count, edge_count, events, edges, families} only; envelope fields excluded; verified by S8 Check 7
Consumer contract	visualization/consumers/ is the ONLY sanctioned interface; never import from compiler/graph/*; JSON schema is the contract
Replay guarantee	evidence_graph.json is self-contained; no compiler state or imports needed to parse it
VERIFICATION absence	S8's verification_complete is always absent by design — S7 writes the file before S8 runs; 0 VERIFICATION-family events is correct, not a defect

Location: evidence_snapshot/<domain>/evidence_graph.json — written by S7, verified by S8.

Contract freeze: The EvidenceQuery public surface is a protocol surface, not a utility library. Additions require explicit versioning. No compiler-internal fields may be exposed.

5.5 Protocol Inspection (pi)

Compilation answers *is this protocol admissible?* Inspection answers *what does this protocol mean?* pi (Protocol Inspection command processor, in pgs_compiler, v0.6.0+) is the read-only query surface over the compiled snapshot set — the relationship-navigation tool that replaces grepping markdown and mentally reconstructing dependency graphs the compiler has already built and verified.

Governing principle (V0): *pi answers questions. The compiler performs changes. The runtime performs execution.* No mutation verb exists in the taxonomy, the CLI, or the library — including caches. Because inspection answers come from the same verified

projections the runtime consumes, a `pi` result carries snapshot authority and is admissible as dossier evidence (Stage 3 impact analysis cites `pi topology impact --json` directly).

One core, three surfaces: the inspection library (`pgs_compiler.inspection: loader → resolver → traversal`) is the core; the terminal CLI/shell and the `--json` output (stable schema, for agents and CI) are its projections. All three return the same answer by construction.

Property	Statement
Read-only	No write API exists anywhere in the inspection path
Workspace explicit	<code>--workspace</code> flag or <code>PGS_WORKSPACE</code> env var; no cwd guessing
Validity gate	Refuses to answer unless <code>snapshot_status.json</code> is <code>VALID</code>
Zero Inference	Full FQDNs required one-shot; bare codes hard-error naming candidates; shell use <code><domain></code> scope is declared and visible in the prompt (<code>pi:<domain>></code>)
Fail hard	Missing projection / unknown FQDN → non-zero exit with explicit cause
Determinism	Same snapshot + same command → byte-identical output
SoC	Zero runtime imports; <code>pi trace explain</code> delegates to <code>pgs_runtime examine</code> via subprocess — <code>pi</code> never parses traces

Query sources (all compiler-materialized): `protocol_snapshot/artifact_index/` (`index.json` — FQDN → domain/structure/kind/paths/addresses; `stores.json` — entity-store ownership join), per-scope `evidence.json` (the artifact-level semantic graph: `NODE_NEXT`, `WF_CONTAINS_NODE`, `CC_BINDS_CT/CS`, `WF_BINDS_RB`, `RB_MAPS`, `GOVERNED_BY` edges), `vocabulary_snapshot/`, `pps_snapshot/index.json`, `protocol_snapshot/behavior_logic/<WF>/*.graph.json`, and `conformance_results.json`. Both index projections enter the workspace only via `pgs_compiler.cli build` — never hand-placed.

Command taxonomy: `pi <object> <verb> [target]` over objects `artifact`, `wf|cc|ct|cs|rb|in|ev|ac`, `topology`, `behavior_logic`, `store`, `vocab`, `pps`, `snapshot`, `trace`, plus top-level `validate` (CI gate: `--strict` exits non-zero unless `VALID` with zero violations) and `stats`. Bare `pi` opens the interactive shell. Full command set: `doc/pgs_cli_cheatsheet.txt`.

6. Runtime Model

6.1 Runtime Constraints (Non-Negotiable)

- Snapshot-only: runtime reads `protocol_snapshot/` exclusively; never touches protocol source
- No discovery: artifact locations come from the snapshot manifest, not filesystem scanning
- No inference: missing binding → hard failure; no fallback implementations
- No domain logic: runtime is semantically blind to what it executes
- Deterministic traversal: graph edges are followed exactly as declared

6.2 Runtime Bindings

- `RB_` maps protocol capability declarations to concrete implementations
- Both `CT_` and `CS_` resolved via the same binding mechanism
- `RB_` resolves implementation location only — behavioral authority originates exclusively from protocol declarations

6.3 Trace Lifecycle

execution start → trace file created at `traces/<domain>/<subdomain>/<TRACE_ID>/<TRACE_ID>.jsonl`

each CC node → structured event appended

execution end → trace sealed; `.md` summary + `.png` visualization written

Trace files are OUTPUT only. Never use as input to compiler, runtime, or other components. Trace IDs are deterministic — same inputs produce the same trace ID. Exit codes: 0 for completed execution (including NACK/VIOLATION), 1 for infrastructure failures.

6.4 CS Substrate Inventory

All `CS_` artifacts live in the `capability_side_effects` domain (owned by `pgs_capabilities`). The side-effect surface is finite and enumerable — this is the entire mutation/IO vocabulary:

CS Substrate	Semantics
<code>CS_REGISTRY_V0</code>	Deduplicated key-value store; ops: REGISTER, RESOLVE, EXISTS, DEREGISTER
<code>CS_MUTABLE_JSON_V0</code>	Read-write JSON state; ops: WRITE, READ, DELETE, EXISTS, LIST_KEYS
<code>CS_APPENDONLY_JSONL_V0</code>	Immutable event stream; append only; never truncated
<code>CS_SEND_EMAIL_V0</code>	Email delivery side effect
<code>CS_WORKFLOW_GATEWAY_V0</code>	Governed sub-workflow invocation — the ONLY way one WF engages another (via a <code>CC_INVOKE_*</code> node)

CS Substrate	Semantics
CS_WORKFLOW_LOOP_V0	Governed bounded iteration over sub-workflow invocations; dispatch mapping declared, never derived at runtime
CS_CONCURRENT_WORKFLOWS_V0	Governed parallel WF dispatch; results correlated by WF FQDN (not array position); aggregate all_succeeded routing
CS_NAME_REGISTRY_V0	Name-scoped registry variant

7. Execution Model

7.1 Governed Execution Topology

PGS execution is **governed declarative graph traversal**, not workflow orchestration. The execution topology is a compile-time governed graph — fully declared before runtime begins, immutable during execution.

TI → IN (admission check)

→ WF (governed topology traversal)

→ CC_1 → CT/CS → outcome (SUCCESS | VIOLATION | ALREADY_EXISTS | ...)

→ CC_2 (inputs resolved via JSONPath: \$.results.<CC_CODE>.<field>)

→ ...

→ TE (boundary projection)

- Topology is a DAG — no cycles, deterministic termination
- All step sequences declared at compile time — runtime is a traversal engine, not an orchestrator
- Outcomes declared in WF_ route traversal — no runtime branching logic
- Runtime receives a fully closed topology graph; it does not discover, repair, or extend it

Workflow composition rules: - WF nodes are IN, CC, and EXIT/EXIT_SUCCESS only — a workflow NEVER appears as a node inside another workflow - Sub-workflow invocation is a gateway CC (CC_INVOKE_* bound to CS_WORKFLOW_GATEWAY_V0); iteration and parallelism use CS_WORKFLOW_LOOP_V0 / CS_CONCURRENT_WORKFLOWS_V0 - EV_ records facts; it never triggers execution — there is no event subscription in the runtime; workflow chaining is always a declared gateway step - A store is written only by CCs of its owning subdomain — cross-subdomain reads/calls are permitted and declared; cross-subdomain writes are forbidden without exception

7.2 CT Purity Invariant

- $\text{Effect}(\text{CT}) = \emptyset$ — transforms have zero side effects
- CT may call other CTs; may never call CS
- CT correctness is the implementation's responsibility; PGS constrains invocation, not logic

7.3 CS Side Effect Boundary

- CS is the only authorized channel for external state mutation
- $\text{MutationSurface} = \{ s : s \in \text{CS_} \}$
- No implicit write path exists anywhere else in the system

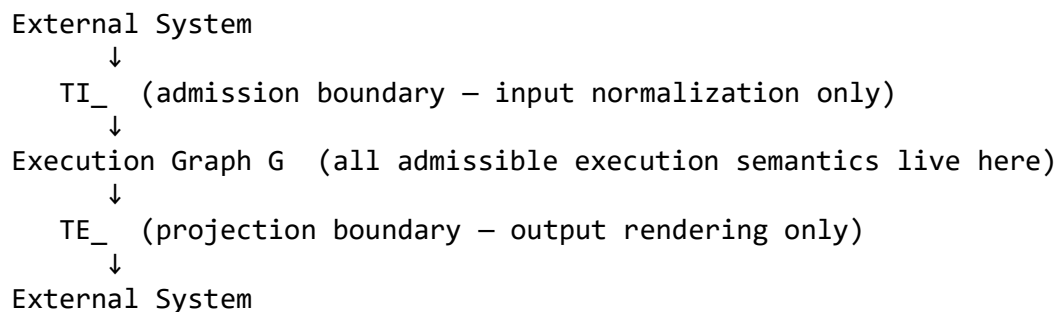
7.4 Trace as Evidence

- Every node execution emits a structured trace event: artifact identity, inputs, outputs, outcome
- Trace is append-only, immutable once written
- “If an action does not appear in the trace, it is architecturally prohibited”
- Replay is deterministic: same trace → same proof

7.5 Ownership Boundaries

Owner	Owns
CC nodes	Input resolution + outcome declaration
CT implementations	Computation logic within declared contract
CS implementations	External IO within declared side-effect scope
Runtime	Graph traversal + trace emission
Compiler	Admissibility determination

7.6 System Boundary Model



TI_ and TE_ are orthogonal boundary concerns — they do not participate in workflow authority or execution orchestration.

7.7 Execution Topology Governance

Doctrine: Execution topology governs traversal structure only.

Topology Property	Statement
Compile-time governed	All step sequences, routing, and input bindings fully declared before runtime
Declarative	Steps are declared; not inferred, not synthesized, not discovered
Immutable after compilation	Compiled topology cannot be modified, extended, or overridden at runtime
Traversal-only scope	Topology governs which capabilities execute in what sequence — nothing else

Topology Property	Statement
Authority orthogonal	Topology may not branch on role, permission, or actor state
Transport orthogonal	Topology may not encode HTTP endpoints, transport conditions, or dispatch logic
Closure required	All step inputs must resolve at compile time; no forward references; no dangling references
No runtime synthesis	Topology may not be generated or inferred from payload content or environment state

Canonical surface governance: Every step's `result_surface` must exactly match the `canonical_surface` declared by the governing `SURFACE_CONTRACT`. Workflow authors MAY route surfaces. Workflow authors MAY NOT define capability semantics.

Topology invariant families (compile-time enforced): - Step identity uniqueness · Capability reference uniqueness · Input reference closure - Routing completeness · Contract closure · Authority orthogonality - Transport orthogonality · Surface canonicity

8. Architectural Properties

The PGS architecture produces eleven named properties. These are not design goals — they are structural outcomes that follow directly from the Governance → Compiler → Runtime separation. Each is derived in full in the technical paper trilogy.

Property	Statement
Implementation Independence	Protocol behavior can be executed without coupling to a specific implementation. The implementation can be replaced entirely without changing protocol behavior. The protocol can evolve entirely without changing the runtime.
Hosting Transparency	The Protocol Snapshot is indifferent to the substrate executing it. A deployment decision changes where execution happens. It cannot change what execution means.
Projection Independence	Governance is separable from execution representation. Multiple projections of the same compiled truth can be produced; selecting one does not alter governed behavior.
Security as Projection Choice	Security posture improves by choosing a more constrained execution projection — without changing the protocol. What the runtime cannot know, an attacker cannot exploit through it.
Runtime Multiplicity	One protocol, one compiled snapshot, many conforming runtimes — cloud, edge, embedded, hardware — all executing identical behavior and producing semantically equivalent traces.
Transport Orthogonality	The workflow topology is constitutionally ignorant of its invocation surface. A workflow executed via CLI and the same workflow executed via REST API are indistinguishable at the topology level.
Structural Parallelism	Concurrency is not engineered into PGS. It is a structural consequence of protocol topology. The runtime exploits independence declared by the topology, not independence created by implementation.
Runtime Stability	The runtime reaches a stable execution contract before the protocol stabilizes. Protocol evolution produces new snapshots. The runtime executes them without modification. New domains do not introduce new runtime code paths.
Trace Portability	Execution evidence is substrate-neutral. Traces produced by a cloud runtime and an embedded firmware runtime executing the same snapshot are semantically equivalent and structurally comparable.
Governance Dividend	As governance surface matures, cost-of-change decreases rather than increases. Governance complexity compounds; execution complexity does not.

Property	Statement
Determinism	Identical inputs produce identical execution paths and identical traces across every conforming runtime on every conforming substrate. This is not engineered into deployments — it is a property of the architecture.

The compiler governs possibility. The runtime governs realization. The separation between them is where Protocol-Governed Systems become governable.

9. Transport Governance

9.1 TI / TE Doctrine

- **TI (Transport Ingress):** Normalizes and validates all external input → canonical internal representation. No business logic.
- **TE (Transport Egress):** Renders internal results for external systems. Boundary projection only. Does not participate in execution semantics.
- Closed-loop invariant: all behavior follows $TI \rightarrow G \rightarrow TE$

9.2 Transport Orthogonality

- Transport substrate (CLI, HTTP, queue, agent) is orthogonal to execution semantics
- Changing from CLI to HTTP does not change the execution graph
- No routing logic in TI/TE — routing lives in WF_ declarations

9.3 Transport Does Not Route

- Transport artifacts normalize and project data only
- Transport may not perform execution routing, orchestration, or behavioral selection
- All routing lives in WF_ DAG outcome edges — never in TI/TE middleware

9.4 Snapshot-Driven Route Loading

- Routes loaded from snapshot; no hardcoded route tables
- Route = (domain, workflow) pair declared in snapshot
- No middleware intelligence — middleware is transport plumbing only

9.5 Transport Federation Doctrine

Transport governance requires Phase Type B federated aggregation to synthesize: route indices · admission maps · projection schemas · transport boundary manifests.

Transport aggregation **may synthesize:** routes · projections · admission maps

Transport aggregation **may NEVER:** mutate execution semantics · alter workflows · inject orchestration · infer routing behavior dynamically

Transport is not federated execution; it is federated boundary governance.

10. Federated Governance

Architectural transition: PGS has evolved from *protocol-governed execution* to *federated governance compilation* — a substantially more powerful architecture class.

10.1 Definition

A **Federated Governance Domain** is a cross-structure governance concern that: - cannot be semantically closed within a single domain structure - requires contribution from multiple domain builds - is compiled via Phase Type B aggregation - produces a governed artifact consumed by execution, transport, or runtime

10.2 Canonical Domain Families

Domain Type	Example	Product	What It Governs
Federated Aggregation	Vocabulary	<code>vocabulary_symbols.js</code> on + semantic index	Protocol ontology; expressible concern surface
Federated Boundary	Transport	route index, admission maps, projection schemas	Boundary admission and projection topology
Federated Admissibility	Authority	admissibility maps, actor eligibility graphs	Execution eligibility and observation rights
Federated Correctness	Conformance (future)	global correctness assertions	Cross-structure invariant enforcement
Federated Topology	Federation (future)	topology graph	Cross-runtime structure relationships

10.3 Invariants for All Federated Domains

Macro-invariant: Federated governance domains may synthesize governance products but may not synthesize execution semantics.

Every federated governance domain must: - be declared via an aggregation `STRUCTURE_` artifact with an explicit `aggregation_type` - declare all contributing source dirs explicitly (`artifact_source_dirs`) - produce deterministic output given the same inputs - run after all contributing Phase Type A builds complete - introduce no execution node types, runtime graph primitives, or middleware chains - preserve orthogonality with the execution layer

10.4 Boundary Orthogonality

Federated governance products may never: mutate execution semantics · alter declared workflow behavior · inject orchestration logic · infer behavioral routing dynamically · become smart runtime components.

Authority governs whether execution may exist. Transport governs where boundaries are. Neither touches how the execution graph runs.

10.5 Architectural Significance

Before	After
Protocol-governed execution	Federated governance compilation
Vocabulary is per-structure output	Vocabulary is a federated ontology product
Transport routes are implicit	Transport routing is a compiled, governed artifact
Authority is runtime middleware	Authority is a compiled admissibility domain
Governance is per-artifact	Governance is a compounding federated product

11. Anti-Patterns

Anti-Pattern	Why It Violates PGS	Correct Approach
Smart runtime	Embeds domain logic in execution layer	Runtime is generic; all behavior in compiled snapshot
Fallback logic	Masks architectural violations; introduces non-determinism	Hard failure; fix the root cause
Dynamic imports	Breaks static resolution; enables runtime code injection	Static imports only; handler registry at compile time
Filesystem inference	Violates zero-inference doctrine	Declare all paths explicitly
Relative path traversal (../)	Path guessing; breaks portability	Script-relative or env-var-driven explicit paths
Editing protocol_snapshot/	Modifies compiler output; overwritten on next build	Change protocol source → recompile
Using trace as input	Trace is evidence, not protocol; breaks separation	Traces are output only
Hardcoded absolute paths	Environment coupling	Declare via env vars or explicit script parameters
CT calling CS	Breaks CT purity invariant	CS calls only from CC-authorized pathways
Middleware routing logic	Transport coupling; breaks transport orthogonality	Routing in WF_ topology only
Role-branching in topology steps	Embeds authority semantics in execution topology	Authority evaluation happens before topology traversal; topology assumes admissibility is resolved
result_surface deviation from canonical	Workflow author redefines what a capability produces	Declare the exact canonical_surface from the governing SURFACE_CONTRACT — routing is permitted, redefinition is not
Runtime step injection	Synthesizes topology from payload or environment	All steps declared at compile time; topology sealed before runtime begins
Event-driven triggering	EV_ is observability/control plane; runtime has no subscription mechanism	Workflow chaining via gateway CC (CC_INVOKE_* on CS_WORKFLOW_GATEWAY_V0); events record the fact

Anti-Pattern	Why It Violates PGS	Correct Approach
WF node inside a WF	Workflows are not node types; embedding breaks topology closure	Sub-workflow invocation via gateway CC; loop/concurrent variants via their CS substrates
Cross-subdomain store write	Violates store ownership boundary	Dependency-gap CC owned by the store's subdomain, declared in the CR (§4.3)
Authoring what already exists	Re-creates baseline capability under a new name; splits identity	Search the snapshot inventory (PPS index) before declaring any new artifact
Environment-driven branching	Implicit behavior; breaks determinism	Explicit outcomes in protocol artifacts
Short-name artifact references	Breaks FQDN identity; causes resolution failures	Always use <code>domain::ARTIFACT_CODE</code>
Package install during operation	Execution environment must be pre-resolved	Resolve all dependencies at bootstrap time
<code>sys.path</code> manipulation	Breaks import isolation	Proper package installs via editable installs

12. Architecture Evolution

Understanding why the architecture looks as it does prevents re-introducing solved problems.

Evolution	What Changed	Why
Short-name → FQDN	All artifact identifiers now require domain: :CODE format	Eliminated ambiguity when multiple domains co-exist in snapshot
Runtime decoupling	Runtime became fully domain-agnostic	Enabled same engine to run blockchain and AI governance without modification
Transport governance formalization	TI/TE defined as distinct concern types	Separated projection semantics from execution semantics; enabled transport orthogonality
CT binding refactor	RB_ now resolves both CT_ and CS_ symmetrically	Eliminated split binding mechanism; runtime is fully implementation-agnostic
Snapshot sovereignty	protocol_snapshot/ declared formally read-only	Prevented drift between compiled and live behavior
TE ontology correction	TE confirmed as boundary projection only; not an execution concern	Clarified that TE has no authority in execution; projection is structural, not behavioral
Federated governance compilation	Compiler gained Phase Type B (cross-structure aggregation); vocabulary was first federated governance product	Single-structure compilation cannot close over cross-structure semantics; vocabulary, transport, and authority require multi-domain synthesis
Protocol-governed execution → federated governance compilation	Architecture class elevated: PGS now governs ontology, routing, and admissibility as compiled artifacts	Governance surfaces compound without runtime coupling; execution remains semantically blind while governance grows richer
Execution topology formalized as governed surface	Execution graph structure elevated from convention to constitutional governance surface with 8 enforced invariants	Topology closure, step identity, canonical surface, authority/transport orthogonality, and runtime synthesis prohibition are now compile-time enforced
Governed declarative graph traversal	PGS execution is no longer adequately described as “workflow orchestration”	Topology is compile-time declared, authority-orthogonal, transport-orthogonal, and immutable after compilation;

Evolution	What Changed	Why
Constitutional Federation	Governance registry organized as <code>registry/FB_*/</code> (14 boundaries); <code>fb.*::</code> is the only valid namespace for governance artifacts	these properties are now constitutional, not conventional Federation boundaries are first-class semantic constructs. Governance sovereignty is physically visible in the registry.
Compiler evidence graph	Compiler gained a governed observability artifact: <code>evidence_graph.json</code> per structure; CAUSALITY + STAGE_SEQUENCE typed edges; SHA-256 hash integrity	Compiler is now self-observable; evidence graph enables visualization, AI tooling, replay, and debugging from a stable JSON schema without importing compiler internals
Governed Evidence System convergence	PGS architecture converges on three compounding properties: Protocol Compiler + Governed Evidence System + Deterministic Execution Fabric	This combination is an unusual architectural category. Cost-of-change decreasing with system growth is repeatedly demonstrated architectural behavior, not an aspirational claim.
Closed-loop governed evolution (v0.5.0)	Change management became a governed concern: <code>FB_CHANGE_MGMT</code> boundary, <code>pgs_change_mgmt</code> repo, staged CR→Mandate pipeline with dossiers, gates, and purity ladder (§4)	Construction and execution were governed; evolution was not. An ungoverned change process accumulates the same rationale decay PGS eliminates elsewhere — the pipeline closes the loop
Orchestration substrate	Workflow composition gained governed primitives: <code>CS_WORKFLOW_GATEWAY_V0</code> , <code>CS_WORKFLOW_LOOP_V0</code> , <code>CS_CONCURRENT_WORKFLOWS_V0</code> ; events confirmed as facts, never triggers	Workflow-to-workflow engagement, iteration, and parallelism are declared side effects — not runtime orchestration logic, not event subscriptions
Subdomain store ownership	A store is written only by its owning subdomain's CCs; peer writes are dependency-gap CCs owned by the peer	Store ownership is a governance boundary; cross-subdomain writes were never legal — the doctrine is now explicit and template-enforced

Evolution	What Changed	Why
Trifecta authoring + Construction Compiler	Authoring became transport-invariant (automated · guided · offline over one worker interface, §4.8); Stage 8 gained a deterministic Construction Compiler that lowers <code>cr_ir/</code> mandates into artifacts under contract authority — candidate capability contracts, propagate-never-invent, TYPED_PORT gaps (§4.10)	Proved the <i>pipeline</i> , not the model, carries the intelligence: a weaker worker needs more gate iterations but converges on the same governed output. Governed authoring properties — Authoring Completeness, Belief Preservation, Projection Fidelity — are now mechanically enforced, not review conventions (§4.9)
Full change-management lifecycle + unified CLI (v0.8.0)	The pipeline gained an explicit lifecycle — DRAFT → CONSTRUCTION_COMPLETE → ADMITTED_UNVALIDATED → EXECUTION_VALIDATED → PROMOTED — fronted by one executable, <code>pgs_change</code> (author · construct · validate · promote, §4.11–4.13); admission and execution validation are distinct gates, and Governance Impact / Promotion ≠ Adoption separate discovery from authority	Change-management output is an admitted protocol delta , not code — and “it compiles” is not “it runs.” Admission proves Protocol Completeness; Execution Validation proves the behaviour runs; Promotion is compiler-gated and rollback-all-on-red. A CR <i>discovers</i> the governance changes it needs but never performs them (§4.12)

Architectural Invariants

Hard constraints. Violation = operational failure or architectural corruption.

Invariant	Statement
FQDN Required	All artifact filenames and references must use domain: :ARTIFACT_CODE — no short names
Snapshot Immutability	protocol_snapshot/ is READ-ONLY at runtime; never edit by hand
No Runtime Discovery	Runtime resolves artifacts from snapshot manifest only; no filesystem scanning
No Execution in Compiler	Compiler never runs CT/CS/domain implementations
No Transport Execution Logic	TI/TE perform normalization and projection only; no business logic
No Hidden State Mutation	All state changes through explicitly declared CS_artifacts
No Ambient Authority	Code has no inherent authority; all authority from (AC, IN, WF, CC) declarations
No Fallback	Missing binding, missing artifact, violated invariant → hard failure
Trace Output Only	Traces written to traces/ by runtime; never used as input to any component
Data Root Explicit	--data-root must be explicitly passed; never inferred
Static Imports Only	No dynamic imports in compiler handlers or operational scripts
No sys.path Manipulation	System path must not be modified in scripts or handlers
Protocol Recompile Required	To change behavior → change protocol source → recompile → rebuild snapshot
CT Purity	CT implementations must be free of side effects; no CS calls from CT
Topology Immutability	Compiled execution topology is immutable; no step added, removed, or rerouted at runtime
Topology Closure	All step inputs must resolve at compile time; no dangling references
No Runtime Topology Synthesis	Execution topology may not be generated or inferred from payload, environment, or runtime state
Topology Traversal Scope	Execution topology governs traversal structure only; may not encode authority or transport semantics

Invariant	Statement
Canonical Surface	Every step's <code>result_surface</code> must exactly match the <code>canonical_surface</code> declared in the governing <code>SURFACE_CONTRACT</code>
Evidence Consumer Isolation	<code>visualization/consumers/</code> must never import from <code>compiler/graph/*</code> ; JSON schema is the only contract
Evidence Hash Contract	<code>evidence_graph_hash</code> covers only <code>{event_count, edge_count, events, edges, families}</code> ; <code>envelope</code> fields excluded
VERIFICATION Family Absent	<code>evidence_graph.json</code> covers S1–S7 by design; 0 VERIFICATION-family events is architecturally correct

Appendix A: Operational Reference

Bootstrap

```
cd pgs_workspace
cd scripts && ./bootstrap_pgs.sh # default: editable installs from ~/pgs/
cd scripts && ./bootstrap_pgs.sh --env remote # published PyPI packages
source .venv/bin/activate
pgs_runtime --help
```

Run Demo End-to-End

```
source .venv/bin/activate
cd scripts && ./demo_sample_workflow.sh
```

Run Workflow (CLI)

```
pgs_runtime run \
  --wf <domain>::<WF_CODE> \
  --payload <path-to-payload.json> \
  --data-root /absolute/path/to/pgs_workspace/data \
  --workspace /absolute/path/to/pgs_workspace
```

Flag	Description
--wf <FQDN>	Workflow to run
--intent <FQDN>	Alternative to --wf; enforces admission gate via declared Intent
--payload <file>	Path to JSON payload file
--data-root <path>	Must be absolute path. Runtime data root; never inferred
--workspace <path>	Workspace root
--rb <FQDN>	Override a specific runtime binding (for testing)
--mode runtime\ authoring	Execution mode (default: authoring)
--debug	Enable debug output

Env var alternatives:

```
export PGS_DATA_ROOT=/absolute/path/to/pgs_workspace/data
export PGS_WORKSPACE=/path/to/pgs_workspace
```

Examine a Trace

```
pgs_runtime examine ./traces/<domain>/<subdomain>/<TRACE_ID>/<TRACE_ID>.jsonl
```

Build (Compiler)

```
# Phase A – per-structure (run in order)
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_PLATFORM_CONFIG
```

```

G_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_BLOCKCHAIN_CONFIG_V0
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_AI_GOVERNANCE_CONFIG_V0

# Cross-structure aggregation happens in `build` (artifact_index/ emission) –
# the former Phase B vocabulary-aggregate compile step is retired; per-domain
# vocabulary is materialized in Phase A (S7).

# Full build – compile + sync + conformance + attestation + snapshot validation
python -m pgs_compiler.cli build --workspace /abs/path/to/pgs_workspace

# Inspect compiled evidence (without recompiling)
python -m pgs_compiler.cli inspect --structure STRUCTURE_BUILD_BLOCKCHAIN_CONFIG_V0 \
    --artifact blockchain::WF_REGISTER_ACTOR_UNVERIFIED_V0
python -m pgs_compiler.cli inspect --structure STRUCTURE_BUILD_BLOCKCHAIN_CONFIG_V0 \
    --family CONSTRUCTION

```

Protocol Inspection (pi)

```

export PGS_WORKSPACE=/abs/path/to/pgs_workspace    # or pass --workspace per command

```

```

# Daily drivers
pi artifact refs blockchain::CC_GENERATE_TX_ID_V0      # who references this artifact
pi topology impact <fqdn> --json                      # transitive consumer closure (dossier evidence)
pi behavior_logic show blockchain::WF_MINT_V0          # execution tree from *.graph.json
pi artifact source blockchain::WF_MINT_V0             # authoring Markdown from PPS snapshot
pi snapshot validate                                  # conformance state of the snapshot
pi validate --strict                                  # CI gate: exit 1 unless VALID + zero violations

```

```

# Interactive shell (tab completion; bare codes resolve within declared scope)
pi
#   pi> use blockchain
#   pi:blockchain> artifact refs CC_GENERATE_TX_ID_V0
#   pi:blockchain> exit

```

Full taxonomy (artifact, kind objects, topology, behavior_logic, store, vocab, pps, snapshot, trace): doc/pgs_cli_cheatsheet.txt. Doctrine: §5.5.

Clean State Rebuild

```
pgs_compiler/scripts/clean_pycache.sh  
pgs_compiler/scripts/clean_outputs_dir.sh  
pgs_compiler/scripts/clean_compiled_artifacts.sh
```

```
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_PLATFORM_CONFIG_V0  
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_BLOCKCHAIN_CONFIG_V0  
python -m pgs_compiler.cli compile --structure STRUCTURE_BUILD_AI_GOVERNANCE_CONFIG_V0  
python -m pgs_compiler.cli build --workspace /abs/path/to/pgs_workspace
```

Unit Tests

```
python pgs_runtime/testbed/run_runtime_tests.py -v
```

HTTP Server

```
scripts/start_http_server.sh
```

See `pgs_cli_cheatsheet.txt` for port overrides and manual invocation options.

Appendix B: Artifact Reference

Snapshot Structure

```
protocol_snapshot/  
  artifacts/  
    workflows/                ← WF_ JSON  
    capability_contracts/     ← CC_ JSON  
    capability_transforms/    ← CT_ JSON  
    capability_side_effects/  ← CS_ JSON  
    runtime_bindings/        ← RB_ JSON  
    intents/                  ← IN_ JSON  
    events/                   ← EV_ JSON  
    actors/                   ← AC_ JSON  
    layers/ invariants/ assertions/ ← governance (compiled from all 14 FB_*  
boundaries)  
    visualization/<WF_NAME>/  
      <WF_NAME>.graph.json    ← machine-readable DAG  
      <WF_NAME>.projection.png ← visual graph  
  
evidence_snapshot/           ← compiler observability (written by S7,  
verified by S8)  
  <domain>/  
    evidence_graph.json      ← semantic causality graph; events + edg  
es + families + hash
```

Both `protocol_snapshot/` and `evidence_snapshot/` are READ-ONLY post-build. Never edit by hand. To change: modify protocol source and recompile.

Trace Structure

```
traces/<domain>/<subdomain>/<TRACE_ID>/  
  <TRACE_ID>.jsonl    ← append-only structured event log (input to pgs_runtim  
e examine)  
  <TRACE_ID>.md       ← human-readable summary  
  <TRACE_ID>.png       ← execution path visualization
```

FQDN Format

`<domain>::<ARTIFACT_CODE>_V<version>`

Examples:

```
blockchain::WF_REGISTER_ACTOR_UNVERIFIED_V0  
ai_governance::WF_GOVERN_AGENT_ACTION_V0  
blockchain::CC_GENERATE_ACTOR_ID_V0  
capability_transforms::CT_PURE_GENERATE_ID_V0
```

Note: CT_ artifacts live in the `capability_transforms` domain.
CS_ artifacts live in the `capability_side_effects` domain.

Governance artifacts use `fb.*` namespace:

```
fb.constitution::CONSTITUTION_FEDERATION_BOUNDARY_V0
fb.topology::CONSTITUTION_EXECUTION_TOPOLOGY_V0
fb.constitution::STRUCTURE_BUILD_PLATFORM_CONFIG_V0
```

Filesystem separator: :: in code → __ (double-underscore) in filenames:

```
blockchain__WF_REGISTER_ACTOR_UNVERIFIED_V0.json
```

Short-code calls to `load_bootstrap_artifact()` without `fb.*::` namespace raise a hard `ValueError` — no fallback.

Execution Outcome Vocabulary

Intent outcomes (IN_gate):

ACK Admission accepted – graph traversal proceeds
NACK Admission rejected – execution halted before graph entry

CC node outcomes:

SUCCESS Normal forward completion
ALREADY_EXISTS Idempotency guard triggered (registry)
VIOLATION Constraint violated (governance, invariant)
DENIED Authorization failed
BACKEND_ERROR Infrastructure or implementation error at CS/CT boundary

Data State Layout

```
data/<domain>/<subdomain>/...      ← one subtree per FB-aligned subdomain
  blockchain/
    identity/           registry/actors.json · events/identity_events.jsonl
    wallet/             state/wallets.json · events/wallet_events.jsonl
    transaction/        state/transactions.json · events/transaction_events.jsonl
1
    mempool/            state/mempool.json · registry/mempool_index.json
    block/              blocks/blocks.json · events/block_events.jsonl
    consensus_pos/      registry/validators.json · rounds/rounds.jsonl · events/
*.jsonl
    orchestration/      state/slot_clock.json · events/simulation_summary.jsonl
    ai_governance/
      ai_licensing/      license_facts.json (read-only seed) · license_registry.j
son · audit_log.jsonl
      agent_governance/ governance_actions.json · governance_audit.jsonl
```

Storage topology is declared in `STRUCTURE` artifacts (`entity_stores`) and resolved via `RB_` — never hardcode paths. Registry stores are deduplicated; `events/` journals are append-only and never truncated; seeds in `seeds/` are source of truth, copied to `data/` by bootstrap.

Appendix C: Workflow Inventory

All available workflows: `ls protocol_snapshot/artifacts/workflows/`

Domain	Subdomain	Workflow Code	Purpose
blockchain	identity	WF_REGISTER_ACTOR_UNVERIFIED_V0	Identity registration
blockchain	identity	WF_VERIFY_ACTOR_V0	Identity verification
blockchain	wallet	WF_CREATE_WALLET_V0	Wallet provisioning (HD keypair derivation)
blockchain	transaction	WF_SUBMIT_TRANSACTION_V0	Generic transaction submission
blockchain	transaction	WF_MINT_V0 · WF_BURN_V0 · WF_TRANSFER_V0	Typed supply/transfer transactions
blockchain	transaction	WF_STAKE_V0 · WF_UNSTAKE_V0 · WF_POOL_V0 · WF_REWARD_V0 · WF_SLASH_V0	Typed staking-economy transactions
blockchain	consensus_pos	WF_REGISTER_VALIDATOR_V0	Validator registration
blockchain	consensus_pos	WF_PROPOSE_BLOCK_V0	Proposer selection → block formation → mempool drain → round record
blockchain	orchestration	WF_RUN_CHAIN_SIMULATION_V0	Top-level simulation: concurrent consensus loop + TX workload
blockchain	orchestration	WF_RUN_CONSENSUS_LOOP_V0	Governed slot-sequence iteration
blockchain	orchestration	WF_RUN_TX_WORKLOAD_V0	Governed TX-sequence iteration (typed dispatch from seed)
blockchain	orchestration	WF_PROCESS_SLOT_V0	One slot: read clock → context → invoke proposal → advance clock
blockchain	—	WF_RUN_CONSENSUS_SLOTS_V0	RETIRED — superseded by WF_RUN_CHAIN_SIMULATION_V0
ai_governance	ai_licensing	WF_PROVISION_AI_LICENSE_V0	AI license provisioning

Domain	Subdomain	Workflow Code	Purpose
ai_governance	ai_licensing	WF_DENY_PROVISION_V0	License denial
ai_governance	ai_licensing	WF_AUTO_RECLAIM_V0	License auto-reclaim
ai_governance	agent_governance	WF_GOVERN_AGENT_ACTION_V0	Agent action governance (7 scenarios)
ai_governance	agent_governance	WF_GOVERN_AGENT_ADMISSION_V0	Agent admission governance
ai_governance	—	WF_DEMO_COLLATZ_CONJECTURE_V0	Domain-neutral execution proof (governed loop)
name_service	—	WF_REGISTER_NAME_V0 · WF_LOOKUP_NAME_V0	Capability-substrate name service

In flight (chain CR, at design approval): WF_INITIALIZE_CHAIN_V0 (genesis bootstrap) · WF_COMMIT_BLOCK_V0 (canonical block commitment).

Payload files: <repo>/testbed/<subdomain>/test_payloads/

```
pgs_runtime run --wf <domain>::<WF_CODE> \
  --payload <domain_repo>/testbed/<subdomain>/test_payloads/<payload>.json \
  --data-root /absolute/path/to/pgs_workspace/data \
  --workspace /absolute/path/to/pgs_workspace
```

Appendix D: Debugging Guide

Common Failure Signatures

Symptom	Root Cause	Fix
ExecutionError: no binding for CT_...	CT declared in protocol; no RB_ entry	Add RB_ binding for that CT
ValidationError: payload rejected at IN_...	Payload violates Intent admission rules	Check IN_ schema; fix payload fields
BuildError: conformance check failed	Protocol artifact violates invariant	Run compiler with --verbose; read assertion output
ALREADY_EXISTS outcome	CS_REGISTRY_V0 found duplicate key	Correct behavior — registry is idempotent by protocol design
Editing protocol_snapshot/ has no effect	Snapshot is compiled output; runtime reads sealed state	Modify protocol source → recompile → rebuild snapshot
SnapshotError: artifact not found	FQDN mismatch or artifact missing from build scope	Check STRUCTURE_ config includes the artifact's domain
Trace shows unexpected routing	Outcome name from CC does not match WF edge	Check CC outcome declarations against WF routing edges
Runtime binding resolution fails	Implementation path changed; RB_ not updated	Update RB_ artifact to point to new implementation

Compile vs Runtime Failures

Type	When Detected	Meaning
Invariant violation	Compile time	Artifact violates constitutional law — cannot enter snapshot
Schema validation failure	Compile time	Artifact malformed — field missing or wrong type
Missing binding	Runtime startup	Snapshot present but RB_ not wired to implementation
Payload rejection	Runtime, at IN_	Input does not satisfy admission conditions
CC outcome mismatch	Runtime, in DAG	Declared outcome not in WF routing table

Trace Debugging

```
pgs_runtime examine ./traces/<domain>/<subdomain>/<TRACE_ID>/<TRACE_ID>.jsonl
# Each event: artifact_id, inputs, outputs, outcome, timestamp
# .md file: human-readable summary
# .png file: visual execution path
```

Snapshot Mismatch

If runtime behavior diverges from expected: recompile and sync. Never patch
protocol_snapshot/ manually — it will be overwritten by next build.

Appendix E: Refactor Patterns

Add a New Artifact Family

1. Define schema in `pgs_governance`
2. Write constitution with invariants
3. Implement assertion handlers (static imports only)
4. Register handlers in handler registry
5. Add `STRUCTURE_config` entry to include new family
6. Recompile

Add a New CT (Capability Transform)

1. Implement deterministic function in `pgs_capabilities` (no side effects)
2. Declare `CT_<NAME>_V0` artifact in the relevant domain repo
3. Add `RB_binding` entry mapping declaration to implementation
4. Reference CT in `CC_capability` contract within a workflow
5. Recompile and sync snapshot

Add a New CS (Capability Side Effect)

1. Implement storage/IO operation in `pgs_capabilities`
2. Declare `CS_<NAME>_V0` artifact in the relevant domain repo (defines semantics, not implementation)
3. Add `RB_binding` entry
4. Reference CS in `CC_` within a workflow
5. Recompile and sync snapshot

Add a New Domain

1. Create new repo (e.g., `pgs_<domain>`)
2. Author protocol artifacts: `WF_`, `CC_`, `IN_`, `EV_` for the domain
3. Add `STRUCTURE_BUILD_<DOMAIN>_CONFIG_V0` in `pgs_compiler`
4. **Add the domain layer entry in `STRUCTURE_DISCOVERY_V0`** — set `registry_module` to the sub-module path (e.g., `pgs_<domain>.registry`), **not** the top-level package. The compiler's `LayerResolver` uses `module_root.parent` to locate the repo root; pointing to the top-level package resolves the wrong parent directory and produces zero compiled artifacts.
5. Wire reusable CT/CS from `pgs_capabilities` via `RB_`
6. Implement any domain-specific CT/CS (rarely needed if library is mature)
7. Compile and sync
8. Add domain to workspace bootstrap and HTTP server config

Add a TI Route (HTTP Endpoint)

1. Declare `TI_` artifact in protocol

2. Add static UI payload directory under domain testbed
3. Reference TI in HTTP server --domain flag
4. Snapshot-driven route registration happens automatically

Extend STRUCTURE Governance

1. Modify STRUCTURE_BUILD_*_CONFIG_V0 artifact in pgs_compiler
2. Add new artifact directories or scope inclusions
3. Recompile — no runtime changes required

Appendix F: Coding Directives

Rule	Rationale
Absolute imports only	No <code>sys.path</code> manipulation; no relative import hacks
Layer isolation	CT cannot call CS; runtime cannot contain domain logic; compiler cannot execute
No fallback logic	Missing artifact = failure; no silent defaults
No dynamic discovery	Artifact locations declared, not scanned
No runtime inference	Runtime resolves from snapshot only; never infers from filesystem
No hidden state mutation	All state changes through declared CS_ only
No hardcoded absolute paths	Use script-relative or env-var-driven paths declared explicitly
FQDN everywhere	<code>domain::ARTIFACT_CODE</code> — no short names, ever
Static imports only	In operational scripts and compiler handlers
Environment pre-resolved	All dependencies resolved at bootstrap time; no installation during operation

Appendix G: Repo Map and Quick Reference

Repository Map

Repo	Layer	Responsibility
pgs_governance	Governance	Constitutional governance, federated boundaries, invariant enforcement
pgs_compiler	Compiler	Compiler pipeline, admissibility construction, conformance generation
pgs_transport	Transport	Ingress/egress adapters for HTTP and CLI surfaces
pgs_runtime	Execution	pgs_runtime CLI; deterministic DAG traversal; snapshot loading
pgs_capabilities	Capability Substrate	Shared CT_ and CS_ implementations; reusable capability library
pgs_blockchain	Domain	Blockchain workflows: identity, wallet, transaction, mempool, consensus_pos, orchestration (chain in flight)
pgs_ai_governance	Domain	AI governance workflows: licensing, agent action, agent admission, reclaim
pgs_change_mgmt	Governed Evolution	Change-management lifecycle (§4): stage templates, dossiers, agent context manifest, engine/ lifecycle services + the pgs_change CLI (author · construct · validate · promote, §4.11–4.13)
pgs_workspace	Entry Point	Compiled snapshot + operational scripts; public developer entry

Dependency direction: pgs_workspace → domains → capabilities → runtime ← compiler ← governance
Change-management direction: pgs_change_mgmt → pgs_compiler (validation) + pgs_workspace (PPS snapshot, read-only)

Artifact Prefix Quick Reference

WF_ Workflow	CC_ Capability Contract
CT_ Transform	CS_ Side Effect
IN_ Intent	RB_ Runtime Binding
EV_ Event	AC_ Actor Context
TI_ Transport Ingress	TE_ Transport Egress
STRUCTURE_ Build Config	

High-Value Grep Patterns

```
# List all workflow artifacts
ls protocol_snapshot/artifacts/workflows/
```

Find binding for a specific CT

```
grep -r "CT_<NAME>" protocol_snapshot/artifacts/runtime_bindings/
```

Find which CC uses a given CS

```
grep -r "CS_<NAME>" protocol_snapshot/artifacts/capability_contracts/
```

Find trace events for a specific CC

```
grep '"artifact_id": "blockchain::CC_<NAME>"' \  
traces/<domain>/<subdomain>/<TRACE_ID>/<TRACE_ID>.jsonl
```

Check snapshot validity marker

```
grep "status" protocol_snapshot/artifacts/workflows/*.json | head -5
```

Research Classification

Doctrine must never be contaminated by unvalidated claims. Every statement derived from an experiment carries a standing classification, and only the first two grades may harden into doctrine:

Grade	Meaning	May enter doctrine?
Validated Finding	Reproduced; mechanism understood; survives adversarial check	Yes — as doctrine
Candidate Invariant	Holds so far; not yet reproduced across domains (n=1)	Yes — flagged provisional
Hypothesis	Plausible, untested	No — track only
Retracted Finding	Was believed, now disproven	No — record the retraction, not the claim
Worker Observation	A trait of the specific worker/harness, not the system	No — benchmark fact, never governance

A claim is promoted only by re-grading, never by repetition. Worker Observations (e.g. a given model's verbosity, discovery weakness, or run-to-run variance) are explicitly out of permanent scope — see §4.7 System Boundary.

Manual Evolution Rule

New content must improve architectural cognition density.

If information is already obvious from source code or protocol artifacts, it does not belong here. If a section does not improve architectural decision quality or cognitive restoration speed, it does not belong here.

PGS prioritizes architectural stability over feature velocity. When architectural tradeoffs occur, preference is given to: explicitness over convenience · governance over heuristics · determinism over flexibility · compile-time enforcement over runtime repair.

End of PGS Field Manual v0 — Doctrine-first. Operations in appendices.